Assignment: Data Importation Using R

Josue Iradukunda

2026-10-25

Contents

1 Introduction

Data importation is an important part of data science because data can be stored in different formats and systems. R provides packages and functions for importing data from statistical software, relational databases, and web APIs.

This assignment demonstrates five methods of data importation:

1. SPSS statistical files.
2. Stata statistical files.
3. SQLite databases.
4. MySQL databases using XAMPP.
5. Web APIs returning JSON data.

The objective is to demonstrate how to import data into R, inspect its structure, display summary statistics, and execute SQL queries where applicable.

2 1. Importing Statistical Data from SPSS

SPSS is statistical software that stores datasets in files with the `.sav` extension. The `haven` package allows R to read these files.

2.1 R code

```
library(haven)

# Locate the sample SPSS file included with haven
file_path <- system.file(
  "examples",
  "iris.sav",
  package = "haven"
)

# Check that the file exists
if (file_path == "") {
  stop("The sample SPSS file was not found.")
}

# Import the SPSS dataset
iris_data <- read_sav(file_path)
```

```r
# Display the first ten rows
print(head(iris_data, 10))
```

```
## # A tibble: 10 x 5
##    Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##           <dbl>       <dbl>        <dbl>       <dbl> <dbl+lbl>
##  1          5.1         3.5          1.4         0.2 1 [setosa]
##  2          4.9         3            1.4         0.2 1 [setosa]
##  3          4.7         3.2          1.3         0.2 1 [setosa]
##  4          4.6         3.1          1.5         0.2 1 [setosa]
##  5          5           3.6          1.4         0.2 1 [setosa]
##  6          5.4         3.9          1.7         0.4 1 [setosa]
##  7          4.6         3.4          1.4         0.3 1 [setosa]
##  8          5           3.4          1.5         0.2 1 [setosa]
##  9          4.4         2.9          1.4         0.2 1 [setosa]
## 10          4.9         3.1          1.5         0.1 1 [setosa]
```

```r
# Display the structure
str(iris_data)
```

```
## tibble [150 x 5] (S3: tbl_df/tbl/data.frame)
##  $ Sepal.Length: num [1:150] 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##   ..- attr(*, "format.spss")= chr "F8.2"
##  $ Sepal.Width : num [1:150] 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##   ..- attr(*, "format.spss")= chr "F8.2"
##  $ Petal.Length: num [1:150] 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##   ..- attr(*, "format.spss")= chr "F8.2"
##  $ Petal.Width : num [1:150] 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##   ..- attr(*, "format.spss")= chr "F8.2"
##  $ Species     : dbl+lbl [1:150] 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
##    ..@ format.spss: chr "F8.0"
##    ..@ labels     : Named num [1:3] 1 2 3
##    .. ..- attr(*, "names")= chr [1:3] "setosa" "versicolor" "virginica"
```

```r
# Display the number of rows and columns
dim(iris_data)
```

```
## [1] 150   5
```

```r
# Display summary statistics
summary(iris_data)
```

```
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width       Species
##  Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100   Min.   :1
##  1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300   1st Qu.:1
##  Median :5.800   Median :3.000   Median :4.350   Median :1.300   Median :2
##  Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199   Mean   :2
##  3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800   3rd Qu.:3
##  Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500   Max.   :3
```

2.2 Explanation

The `read_sav()` function imports an SPSS dataset into R. The `head()` function displays the first rows, `str()` shows the variables and their data types, `dim()` returns the number of rows and columns, and `summary()` provides descriptive statistics.

2.3 Result

The dataset is imported into R and its contents, structure, dimensions, and summary statistics are displayed in the report.

3 2. Importing Statistical Data from Stata

Stata is statistical software that stores datasets in files with the `.dta` extension. The `haven` package supports reading and writing Stata files.

In this example, a small student dataset is created, exported to Stata format, and imported back into R.

3.1 R code

```
library(haven)

# Create sample student data
students <- data.frame(
  Student_ID = c(1, 2, 3, 4, 5),
  Name = c("Alice", "Bob", "Claudine", "David", "Eric"),
  Age = c(21, 23, 22, 24, 20),
  Score = c(85, 78, 92, 88, 75)
)

# Export data to Stata format
write_dta(students, "students.dta")

# Import the Stata dataset into R
stata_data <- read_dta("students.dta")

# Display the imported data
print(stata_data)
```

```
## # A tibble: 5 x 4
##   Student_ID Name       Age Score
##        <dbl> <chr>    <dbl> <dbl>
## 1          1 Alice       21    85
## 2          2 Bob         23    78
## 3          3 Claudine    22    92
## 4          4 David       24    88
## 5          5 Eric        20    75
```

```
# Display the structure
str(stata_data)
```

```
## tibble [5 x 4] (S3: tbl_df/tbl/data.frame)
##  $ Student_ID: num [1:5] 1 2 3 4 5
##   ..- attr(*, "format.stata")= chr "%10.0g"
##  $ Name      : chr [1:5] "Alice" "Bob" "Claudine" "David" ...
##   ..- attr(*, "format.stata")= chr "%-9s"
##  $ Age       : num [1:5] 21 23 22 24 20
##   ..- attr(*, "format.stata")= chr "%10.0g"
##  $ Score     : num [1:5] 85 78 92 88 75
##   ..- attr(*, "format.stata")= chr "%10.0g"
```

```r
# Display the number of rows and columns
dim(stata_data)
```

```
## [1] 5 4
```

```r
# Display summary statistics
summary(stata_data)
```

```
##    Student_ID        Name        Age         Score
##  Min.   :1    Length   :5   Min.   :20   Min.   :75.0
##  1st Qu.:2    N.unique :5   1st Qu.:21   1st Qu.:78.0
##  Median :3    N.blank  :0   Median :22   Median :85.0
##  Mean   :3    Min.nchar:3   Mean   :22   Mean   :83.6
##  3rd Qu.:4    Max.nchar:8   3rd Qu.:23   3rd Qu.:88.0
##  Max.   :5                  Max.   :24   Max.   :92.0
```

3.2 Explanation

The `write_dta()` function saves the student dataset in Stata format. The `read_dta()` function then imports the saved file into R.

This demonstrates how R can exchange data with Stata-compatible files.

3.3 Result

The imported dataset contains five students and four variables: student ID, name, age, and score.

4 3. Importing Data from SQLite

SQLite is a relational database management system that stores its data in a database file. Unlike MySQL, it does not require a separate database server.

The `DBI` and `RSQLite` packages are used to create the database, write data into a table, retrieve records, and execute SQL queries.

4.1 R code

```r
library(DBI)
library(RSQLite)

# Create or connect to the SQLite database
con_sqlite <- dbConnect(
  SQLite(),
  "banking_database.sqlite"
)

# Create sample customer data
customers <- data.frame(
  customer_id = c(1, 2, 3, 4),
  name = c("Alice", "Bob", "Claudine", "David"),
  age = c(25, 30, 28, 35),
  balance = c(1000, 1500, 20000, 1200)
)

# Write the data into a database table
```

```r
dbWriteTable(
  con_sqlite,
  "customers",
  customers,
  overwrite = TRUE
)

# Display available tables
print(dbListTables(con_sqlite))
```

```
## [1] "customers"
```

```r
# Retrieve all customer records
imported_data <- dbReadTable(
  con_sqlite,
  "customers"
)

print(imported_data)
```

```
##   customer_id     name age balance
## 1           1    Alice  25    1000
## 2           2      Bob  30    1500
## 3           3 Claudine  28   20000
## 4           4    David  35    1200
```

```r
# Query customers whose balance exceeds 10000
query_result <- dbGetQuery(
  con_sqlite,
  "SELECT name, balance
   FROM customers
   WHERE balance > 10000"
)

# Display the query result
print(query_result)
```

```
##       name balance
## 1 Claudine   20000
```

```r
# Close the database connection
dbDisconnect(con_sqlite)
```

4.2 Explanation

The `dbConnect()` function establishes a connection to the SQLite database. The `dbWriteTable()` function stores customer data in a table named `customers`.

The `dbReadTable()` function retrieves the table into R, while `dbGetQuery()` executes an SQL statement to select customers whose balances exceed 10,000.

4.3 Result

The query returns Claudine, whose balance is 20,000. The complete customer table is also displayed.

5 4. Importing Data from MySQL Using XAMPP

MySQL is a relational database management system that can be run locally using XAMPP. In this example, R connects to an existing database named `wpdbase`.

The `DBI` package provides database functions, while `RMariaDB` provides the connection driver.

Prerequisites: Start MySQL in XAMPP and ensure that the `wpdbase` database contains a table named `department`.

5.1 R code

```
library(DBI)
library(RMariaDB)

# Connect to the existing MySQL database
con_mysql <- dbConnect(
  RMariaDB::MariaDB(),
  host = "127.0.0.1",
  port = 3306,
  username = "root",
  password = "",
  dbname = "wpdbase"
)

# Check whether the connection is valid
print(dbIsValid(con_mysql))
```

```
## [1] TRUE
```

```
# Display all tables in the database
tables <- dbListTables(con_mysql)
print(tables)
```

```
## [1] "assignment" "department" "employee"   "project"
```

```
# Check whether the department table exists
if (dbExistsTable(con_mysql, "department")) {

  # Import the department table into R
  dept_data <- dbReadTable(
    con_mysql,
    "department"
  )

  # Display the first six rows
  print(head(dept_data))

  # Display the dimensions
  print(dim(dept_data))

  # Retrieve up to five records using SQL
  query_result <- dbGetQuery(
    con_mysql,
    "SELECT * FROM department LIMIT 5"
  )
```

```
  print(query_result)

} else {
  cat(
    "The department table was not found in wpdbase.\n",
    "Check the table name and available tables.\n"
  )
}
```

```
##    DepartmentName BudgetCode OfficeNumber DepartmentPhone
## 1      Accounting  BC-500-10   BLDG01-120    360-285-8405
## 2  Administration  BC-100-10   BLDG01-210     30-285-8100
## 3         Finance  BC-400-10   BLDG01-110    360-285-8400
## 4 Human Resources  BC-300-10   BLDG01-230    360-285-8300
## 5     InfoSystems  BC-700-10   BLDG02-210    360-285-8600
## 6           Legal  BC-200-10   BLDG01-220    360-285-8200
## [1] 9 4
##    DepartmentName BudgetCode OfficeNumber DepartmentPhone
## 1      Accounting  BC-500-10   BLDG01-120    360-285-8405
## 2  Administration  BC-100-10   BLDG01-210     30-285-8100
## 3         Finance  BC-400-10   BLDG01-110    360-285-8400
## 4 Human Resources  BC-300-10   BLDG01-230    360-285-8300
## 5     InfoSystems  BC-700-10   BLDG02-210    360-285-8600
```

```
# Close the database connection
dbDisconnect(con_mysql)
```

5.2 Explanation

The `dbConnect()` function connects R to the MySQL-compatible server running on the local computer.

The `dbListTables()` function lists the available tables. The `dbReadTable()` function imports the `department` table, and `dbGetQuery()` retrieves up to five records using SQL.

5.3 Result

If the database connection succeeds and the `department` table exists, the report displays the imported data, its dimensions, and the query results.

If the table does not exist, a message is displayed instead.

Note: The connection settings above assume a local XAMPP configuration with username `root` and an empty password. Change them if your MySQL setup uses different credentials. Never publish database passwords in a public repository.

6 5. Importing Data from a Web API

A web API allows applications to exchange data over the internet. Many REST APIs return data in JSON format.

This example uses JSONPlaceholder, a free testing API that provides sample posts.

The `httr2` package sends the HTTP request and retrieves the response.

6.1 R code

```r
library(httr2)

# Define the API endpoint
api_url <- "https://jsonplaceholder.typicode.com/posts"

# Send the HTTP request
response <- request(api_url) |>
  req_perform()

# Display the HTTP status code
status_code <- resp_status(response)
print(status_code)
```

```
## [1] 200
```

```r
# Convert JSON data into an R data frame
api_data <- resp_body_json(
  response,
  simplifyVector = TRUE
)

# Display the first six records
print(head(api_data))
```

```
##   userId id
## 1      1  1
## 2      1  2
## 3      1  3
## 4      1  4
## 5      1  5
## 6      1  6
##                                                                          title
## 1 sunt aut facere repellat provident occaecati excepturi optio reprehenderit
## 2                                                               qui est esse
## 3               ea molestias quasi exercitationem repellat qui ipsa sit aut
## 4                                                    eum et est occaecati
## 5                                                      nesciunt quas odio
## 6                                          dolorem eum magni eos aperiam quia
##
## 1                                                quia et suscipit\nsuscipit recusandae consequuntur
## 2 est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanc
## 3                                          et iusto sed quo iure\nvoluptatem occaecati omnis eligen
## 4               ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident
## 5                                                repudiandae veniam quaerat sunt sed\nali
## 6            ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molesti
```

```r
# Display the structure
str(api_data)
```

```
## 'data.frame':    100 obs. of  4 variables:
##  $ userId: int  1 1 1 1 1 1 1 1 1 1 ...
##  $ id    : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ title : chr  "sunt aut facere repellat provident occaecati excepturi optio reprehenderit" "qui est
##  $ body  : chr  "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit mo
```

```r
# Display the number of rows and columns
dim(api_data)
```

```
## [1] 100   4
```

```r
# Display summary information
summary(api_data)
```

```
##      userId           id              title              body
##  Min.   : 1.0   Min.   :  1.00   Length   :100   Length   :100
##  1st Qu.: 3.0   1st Qu.: 25.75   N.unique :100   N.unique :100
##  Median : 5.5   Median : 50.50   N.blank  :  0   N.blank  :  0
##  Mean   : 5.5   Mean   : 50.50   Min.nchar: 12   Min.nchar:102
##  3rd Qu.: 8.0   3rd Qu.: 75.25   Max.nchar: 79   Max.nchar:225
##  Max.   :10.0   Max.   :100.00
```

6.2 Explanation

The `request()` function defines the API endpoint, while `req_perform()` sends the HTTP request.

The `resp_status()` function checks the HTTP status code. A status code of `200` indicates a successful response.

The `resp_body_json()` function converts the returned JSON data into an R object. With `simplifyVector = TRUE`, the regular tabular response can be represented as a data frame.

6.3 Result

The API normally returns 100 posts, with four variables: `userId`, `id`, `title`, and `body`.

An active internet connection is required to run this example.

7 6. Comparison of the Five Data Import Methods

Data source	Format or system	R package or function	Main purpose
SPSS	`.sav`	`haven::read_sav()`	Import statistical data
Stata	`.dta`	`haven::read_dta()`	Import Stata datasets
SQLite	SQLite database file	`DBI`, `RSQLite`	Create tables and retrieve data
MySQL	MySQL database server	`DBI`, `RMariaDB`	Connect to an existing database
Web API	JSON over HTTP	`httr2`	Retrieve data from an online service

8 7. Conclusion

This assignment demonstrates five practical methods of importing data into R.

First, the `haven` package was used to import data from SPSS and Stata files. Second, the `DBI` interface and database drivers were used to create and retrieve data from SQLite and connect to an existing MySQL database. Finally, the `httr2` package was used to retrieve JSON data from a web API.

These techniques are important in data science because data is often stored across multiple formats, databases, and online services. Understanding how to import and inspect these datasets provides a foundation for further data cleaning, analysis, visualization, and statistical modeling.

9 References

1. R Project: https://www.r-project.org/
2. Haven package: https://haven.tidyverse.org/
3. DBI package: https://dbi.r-dbi.org/
4. RMariaDB package: https://rmariadb.r-dbi.org/
5. RSQLite package: https://rsqlite.r-dbi.org/
6. httr2 package: https://httr2.r-lib.org/
7. SQLite: https://www.sqlite.org/
8. JSONPlaceholder API: https://jsonplaceholder.typicode.com/